30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 14

Data Layer Architecture — Part 2: Offline-First, Sync & Scale



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 14

Why offline-first is Day 14

Day 13 designed repositories, models and caches. Part 2 is where designs break in production: **writes without a network**, two devices editing the same record, and schema changes shipped to devices you can't roll back.

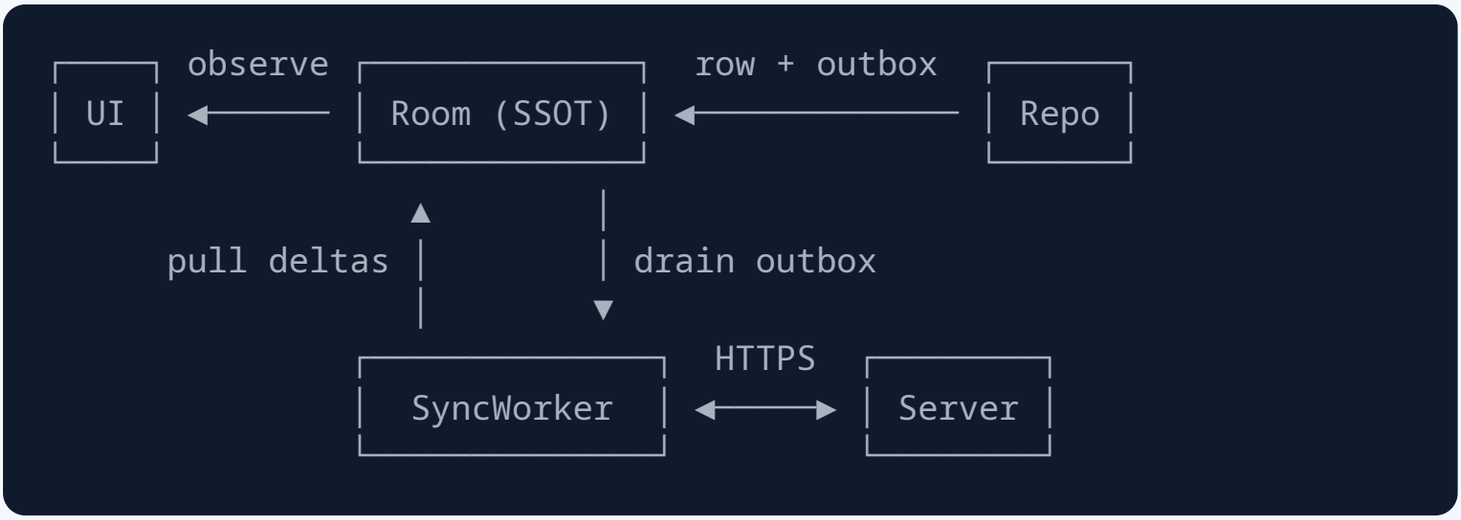
- Offline reads are easy — **offline writes** and **sync** are the hard part
- Conflicts: detecting them and resolving them per data type
- Paging large lists from network **and** database with one source of truth
- Migrations and encryption at rest for data that must survive years
- One data layer for Android and iOS with Kotlin Multiplatform

Level	Questions
Junior	What offline-first means · Offline writes · Why WorkManager for sync
Mid	The outbox pattern · The sync worker · Pulling changes · Paging with RemoteMediator
Senior	Detecting conflicts · Resolving conflicts · Migrations at scale
Architect	Encryption at rest · KMP data layer · Sync for money



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: local first, sync in the background



- **The UI reads only the local database** — online or offline, same code path
- **Writes land locally first**, together with an outbox entry, in one transaction
- **A background worker syncs**: pushes the outbox, pulls server changes, resolves conflicts

TWO TRUTHS, TWO LEVELS

On the device, the database is the single source of truth **for the UI**. Across devices, the **server** is the authority that decides how conflicting writes end.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 14

Junior

What offline-first means · Offline writes · Why WorkManager for sync



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What does offline-first actually mean?

Android's definition: an app that can perform **all, or a critical subset** of its core functionality without internet access.

	Online app with a cache	Offline-first app
Reads	Network; cache as fallback	Always the local database
Writes	Fail without network	Saved locally, synced later
Network	On the critical path	In the background
Loading states	Spinner per request	Content first, sync status on the side

Decide **which** features must work offline — offline-first is a per-feature product decision (Day 13, Q12).



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What interviewers check

INTERVIEW TRAP

"Offline-first means caching API responses." — A cache helps you read. Offline-first also means **accepting writes** without a network and reconciling them later — that's where the real design work is.

INTERVIEW TRAP

"Check connectivity before each request." — "Network available" doesn't mean your server is reachable (captive portals, VPNs, outages). Try the request, handle failure, and let WorkManager's network constraint decide when to retry. Use connectivity callbacks only for UX hints.

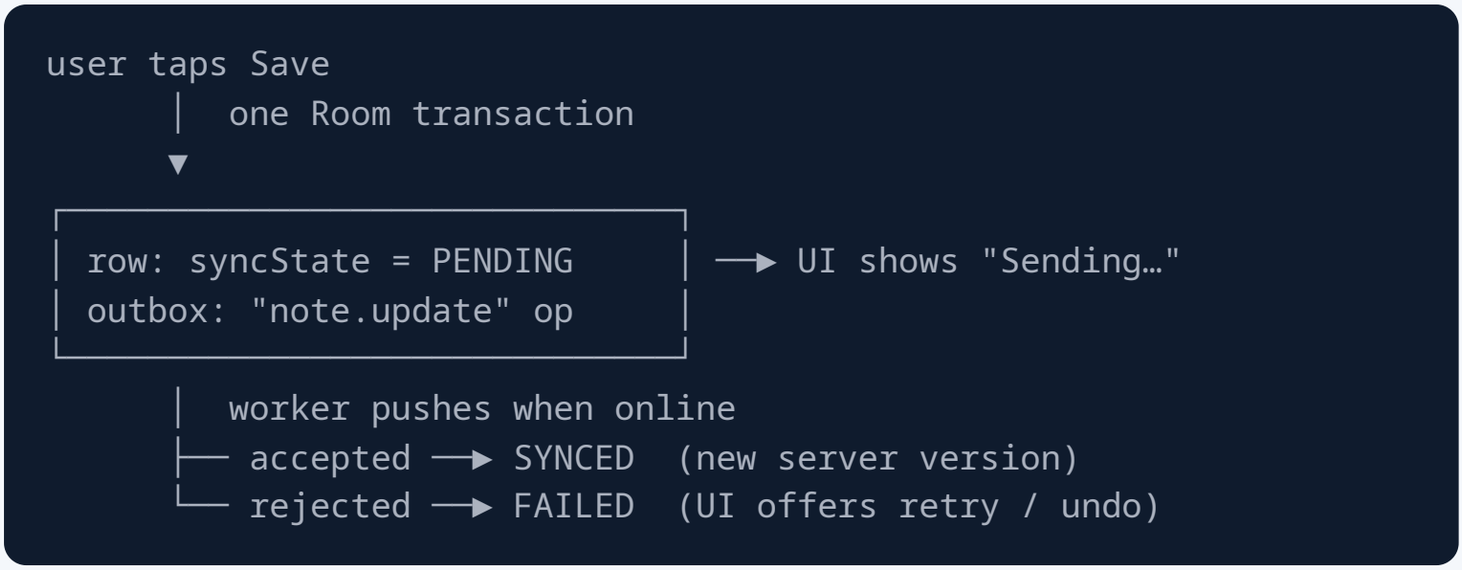
FOLLOW-UP

"If the server is the authority, how is the local DB the source of truth?" — Different levels: the UI observes **only** the DB, so the screen is always consistent; the server resolves **conflicts** and its result is written back into the DB.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. The life of an offline write



- The user sees their change **immediately** — it's already in the database
- Sync state is **data**: store it in the row, render it like any other field
- A rejected write must be visible; silently reverting it is data loss from the user's view



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Where optimistic writes go wrong

INTERVIEW TRAP

"Optimistic UI = update the ViewModel state and call the API." — Process death or a failed request loses the change. Persist first, then sync: the database, not the ViewModel, holds the pending edit.

INTERVIEW TRAP

"Disable the button while offline." — That's an online app with extra steps. If the feature is on your offline list, accept the write and show its sync state.

FOLLOW-UP

"Which IDs do records created offline get?" — Client-generated IDs (UUIDs), so related rows and retries can reference the record before the server has seen it. No temporary IDs to rewrite later.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Why WorkManager runs the sync

```
class SyncScheduler(private val workManager: WorkManager) {
    private val online = Constraints.Builder()
        .setRequiredNetworkType(NetworkType.CONNECTED)
        .build()

    fun requestSync() {
        val request = OneTimeWorkRequestBuilder<SyncWorker>()
            .setConstraints(online)
            .build()
        workManager.enqueueUniqueWork(
            "sync",
            ExistingWorkPolicy.APPEND_OR_REPLACE,
            request,
        )
    }
}
```

- Work is **persisted**: it survives process death and reboots
- Runs when the **constraint** is met; `Result.retry()` uses backoff (exponential by default)
- **Unique work**: many writes → one sync queue, not ten parallel syncs



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. The WorkManager traps

INTERVIEW TRAP

"Sync only on unmetered networks" (v1 called this ideal). — For **user writes** that can mean "never" for people on mobile data. Use `CONNECTED` for user changes; keep `UNMETERED` for bulk prefetch.

INTERVIEW TRAP

`ExistingWorkPolicy.KEEP` for write-triggered sync. A request that arrives while the worker is finishing is dropped, and the new change waits for the next trigger. `APPEND_OR_REPLACE` queues another run after the current one.

INTERVIEW TRAP

"WorkManager guarantees immediate execution." — It guarantees **eventual** execution when constraints allow. The system can defer it, and a force-stop cancels work until the app is opened again. Expedited work, Doze and standby buckets → Day 15.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 14



Mid-Level

The outbox pattern · The sync worker · Pulling changes · Paging with RemoteMediator



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. The outbox pattern

```
suspend fun addComment(postId: String, text: String) {
    val id = UUID.randomUUID().toString()
    val payload = json.encodeToString(
        NewCommentPayload(id, postId, text),
    )
    db.withTransaction {
        commentDao.upsert(
            CommentEntity(id, postId, text, SyncState.PENDING),
        )
        outboxDao.insert(
            OutboxEntry(opId = id, type = "comment.create",
                payload = payload),
        )
    }
    syncScheduler.requestSync()
}
```

The row and the outbox entry are written **atomically**: there is never a pending comment without a queued operation, or the reverse.



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Designing the outbox

```
@Entity(tableName = "outbox")
data class OutboxEntry(
    @PrimaryKey(autoGenerate = true) val seq: Long = 0,
    val opId: String,          // idempotency key (header)
    val type: String,          // "comment.create", "note.update"
    val payload: String,       // versioned JSON
    val attempts: Int = 0,
)
```

- **Idempotency key:** the server stores it; a retried request returns the first result instead of creating a duplicate
- **Order:** drain by `seq` — an edit must not reach the server before the create
- **Coalesce** when safe: five edits to one note can become one update

INTERVIEW TRAP

"Retries are safe because we use WorkManager." — WorkManager retries the **worker**; it doesn't know whether the server applied the request before the connection dropped. Only an idempotency key makes a retry safe.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. The sync worker

```
@HiltWorker
class SyncWorker @AssistedInject constructor(
    @Assisted ctx: Context,
    @Assisted params: WorkerParameters,
    private val outbox: OutboxDao,
    private val api: SyncApi,
) : CoroutineWorker(ctx, params) {
    override suspend fun doWork(): Result {
        while (true) {
            val op = outbox.oldest() ?: break
            when (api.push(op)) {
                PushResult.Accepted -> outbox.complete(op)
                PushResult.Rejected -> outbox.fail(op)
                PushResult.Transient -> return Result.retry()
            }
        }
        // then pull server changes (Q6)
        return Result.success()
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Rules for a worker that never corrupts data

- `complete(op)`: mark the row `SYNCED` **and** delete the entry in one transaction
- `push` maps errors (Day 13's `safeCall`): offline/5xx → `Transient`, validation 4xx → `Rejected`
- **Push, then pull:** the server's answer already contains your accepted changes
- Cap retries with `runAttemptCount` and surface a "can't sync" state instead of retrying forever

INTERVIEW TRAP

Returning `Result.failure()` on a network error. The work is finished for good — pending changes sit in the outbox until some other trigger. Transient problems → `Result.retry()`.

INTERVIEW TRAP

Retrying a rejected (4xx) operation forever. That "poison message" blocks every operation behind it. Mark it `FAILED`, tell the user, and keep draining the queue.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Pulling changes: deltas, not full downloads

```
suspend fun pullChanges() {
    var cursor = syncMetaDao.cursor()  // null = full sync
    do {
        val page = api.changes(since = cursor)
        db.withTransaction {
            noteDao.upsertIfNotPending(
                page.upserts.map { it.toEntity() },
            )
            noteDao.deleteByIds(page.deletedIds)
            syncMetaDao.setCursor(page.nextCursor)
        }
        cursor = page.nextCursor
    } while (page.hasMore)
}
```

- The server returns only what changed since a **cursor** (a change-list version or token)
- Data **and** the new cursor are committed in one transaction — a crash never skips a change
- Now in Android uses the same idea: change lists with versions per data type



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. What breaks delta sync

INTERVIEW TRAP

Keeping the cursor in DataStore while the data lives in Room. Two stores, no shared transaction: a crash after saving the cursor but before the rows skips those changes **forever**. Store the cursor in the same database.

INTERVIEW TRAP

Forgetting deletions. If the server only sends changed rows, records deleted on the server live on every device. The change feed needs **tombstones** (deleted IDs).

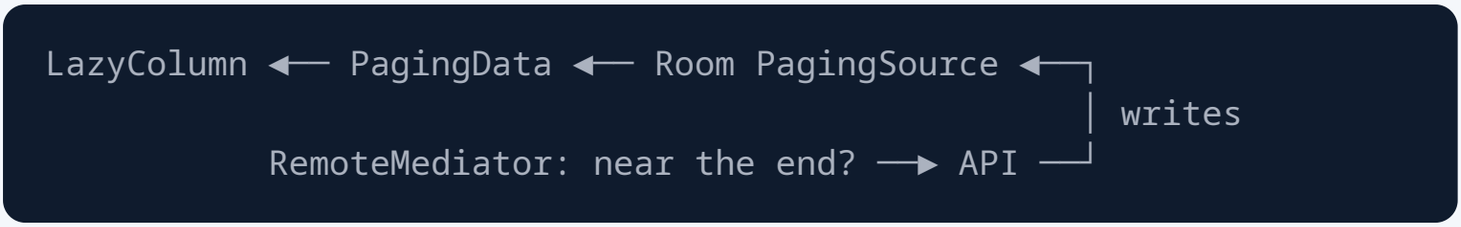
FOLLOW-UP

"What if the cursor is too old?" — The server may no longer keep that history. It should answer "full resync required"; the client re-downloads while keeping pending local writes in the outbox.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Paging 3 with RemoteMediator




```kotlin
@OptIn(ExperimentalPagingApi::class)
fun articles(): Flow<PagingData<Article>> = Pager(
    config = PagingConfig(pageSize = 30),
    remoteMediator = ArticleRemoteMediator(db, api),
    pagingSourceFactory = { db.articleDao().pagingSource() },
).flow.map { page -> page.map { it.toDomain() } }
```

- The UI pages **only from Room**; the mediator loads network pages **into** Room when the list runs out
- Offline, the list still scrolls through everything already stored
- In the ViewModel: `.cachedIn(viewModelScope)`



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Inside the mediator

```
override suspend fun load(
    loadType: LoadType,
    state: PagingState<Int, ArticleEntity>,
): MediatorResult {
    val cursor = when (loadType) {
        LoadType.REFRESH -> null
        LoadType.PREPEND -> return MediatorResult.Success(true)
        LoadType.APPEND -> keys.nextCursor()
            ?: return MediatorResult.Success(true)
    }
    return try {
        val page = api.articles(cursor, state.config.pageSize)
        db.withTransaction {
            if (loadType == LoadType.REFRESH) dao.clearFeed()
            keys.save(page.nextCursor)
            dao.insertAll(page.items.map { it.toEntity() })
        }
        MediatorResult.Success(page.nextCursor == null)
    } catch (e: IOException) { MediatorResult.Error(e) }
}
```

INTERVIEW TRAP

Clearing the table on refresh **outside** the transaction — the list flashes empty. And the paging query needs a stable `ORDER BY` (e.g. a server position column); rows have no guaranteed insertion order.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 14



Senior

Detecting conflicts · Resolving conflicts · Migrations at scale



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Detecting conflicts

```
@Entity(tableName = "notes")
data class NoteEntity(
    @PrimaryKey val id: String,  // client-generated UUID
    val title: String,
    val body: String,
    val baseVersion: Long,       // server version we edited
    val syncState: SyncState,    // SYNCED, PENDING, FAILED
)
```

```
PUT /notes/42   If-Match: "7"
<- 200 OK  ETag: "8"          nobody else changed it
<- 412 Precondition Failed    server is at "9": conflict
```

Optimistic concurrency: the client sends the version it started from; the server applies the write only if it's still current. Many APIs answer a mismatch with `409 Conflict` instead.



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Why versions, not clocks

- The **server** assigns versions (an integer or ETag) — one ordering for everyone
- Keep `baseVersion` from the moment the edit started, not from the last pull
- A pull must not overwrite a `PENDING` row — that silently discards the user's edit

INTERVIEW TRAP

"Last-write-wins by comparing timestamps" (v1's default). — Device clocks drift and users change them; the "latest" edit can be the oldest one. Order by server-assigned versions, or by timestamps the server stamps.

INTERVIEW TRAP

No conflict detection at all. Every write is a blind overwrite — the conflict still happens, you just never find out whose data was lost.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Resolving conflicts — per data type

Strategy	Result	Fits
Server wins	Local edit discarded (and shown)	Server-computed data, prices, permissions
Client wins	Overwrites the server	Single-user data, device settings
Last-write-wins (server order)	Latest accepted write stays	Single values: display name, "last seen"
Field-level three-way merge	Non-overlapping fields both kept	Profiles, forms, notes metadata
Operations, not state	"+1 item" applied on the server	Counters, carts, likes
CRDTs	Converges automatically	Collaborative editing
Ask the user	User picks or merges	Documents where both edits matter



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. A three-way merge

```
fun <T> merge3(base: T, local: T, remote: T): T = when {
    local == base -> remote      // only the server changed
    remote == base -> local      // only we changed
    else -> remote               // true conflict: policy
}

fun mergeNote(base: Note, local: Note, remote: Note) =
    remote.copy(
        title = merge3(base.title, local.title, remote.title),
        body = merge3(base.body, local.body, remote.body),
    )
```

Keep the **base** (the last synced version) to tell "I changed it" from "they changed it".

INTERVIEW TRAP

"Use last-write-wins for chat messages" (v1's example). — Messages are **append-only**: each has its own client-generated ID, so two messages never conflict. LWW is for a single value that both sides overwrite.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Migrations at scale: write them

```
val MIGRATION_7_8 = object : Migration(7, 8) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL(
            """CREATE TABLE orders_new (
                id TEXT NOT NULL PRIMARY KEY,
                total_cents INTEGER NOT NULL)""",
        )
        db.execSQL(
            """INSERT INTO orders_new (id, total_cents)
               SELECT id, CAST(ROUND(total * 100) AS INTEGER)
               FROM orders""",
        )
        db.execSQL("DROP TABLE orders")
        db.execSQL("ALTER TABLE orders_new RENAME TO orders")
    }
}
```

The copy-table recipe works on every SQLite version: create, copy with **one set-based** `INSERT … SELECT`, drop, rename — then recreate indexes.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Migrations at scale: test them

```
@get:Rule
val helper = MigrationTestHelper(
    InstrumentationRegistry.getInstrumentation(),
    AppDatabase::class.java,
)

@Test fun migrate7To8_convertsMoney() {
    helper.createDatabase(DB, 7).apply {
        execSQL("INSERT INTO orders VALUES ('a', 12.5)")
        close()
    }
    val db = helper.runMigrationsAndValidate(
        DB, 8, true, MIGRATION_7_8,
    )
    db.query("SELECT total_cents FROM orders").use { c ->
        c.moveToFirst()
        assertEquals(1250L, c.getLong(0))
    }
}
```

`runMigrationsAndValidate` compares the result with the exported schema — the same check Room runs on real devices.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Rules for millions of devices

- `exportSchema = true` and commit the schema JSON (Room Gradle plugin: `room { schemaDirectory(…) }`) — v1's example had it `false`
- **Users skip versions:** Room chains 3→4→…→8, so every old step stays in the code and in tests
- Migrations run when the database is **first opened** — a slow one delays the first screen; use set-based SQL, not Kotlin loops over rows
- `AutoMigration` for simple changes; renames and deletes need a spec (`@RenameColumn`, `@DeleteColumn`); data transforms need a manual `Migration`

INTERVIEW TRAP

Destructive fallback "just in case". For a pure cache it's fine; for user data it silently wipes drafts and pending outbox entries on every failed migration. Test the migration instead.

INTERVIEW TRAP

A migration that runs but produces a slightly different schema (a missing `NOT NULL`, a forgotten index). Room's validation throws when the database opens — on every launch. That crash loop is why migration tests are not optional.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 14



Architect

Encryption at rest · KMP data layer · Sync for money



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Encryption at rest — what each layer buys you

Layer	Protects against	Doesn't protect against
Device encryption (file-based, standard on modern devices)	A lost or stolen phone that is locked	Anything that runs while it's unlocked
App sandbox (private files, databases)	Other apps	Root, un-excluded backups
App-level encryption with a Keystore-held key (SQLCipher, Tink)	Copied files: extraction, backups, forensic dumps	Code running as your app on a compromised device
Key requires user authentication	Using data without the user present	Nothing is free: UX cost, keys can be invalidated (Day 20)

Keystore keys are **non-exportable**, but a compromised device can still ask the Keystore to *use* them. Encryption raises the cost of an attack; it doesn't make the device trusted.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Encrypting Room, and the traps

```
System.loadLibrary("sqlcipher")
val passphrase: ByteArray = keyManager.dbPassphrase()
val db = Room
    .databaseBuilder(ctx, AppDatabase::class.java, "app.db")
    .openHelperFactory(SupportOpenHelperFactory(passphrase))
    .build()
```

- Passphrase: random bytes generated once, stored **wrapped by a Keystore key** — never hard-coded
- Files and DataStore: encrypt the bytes with **Tink** (keyset wrapped by the Keystore). Secure storage in depth → Day 19

INTERVIEW TRAP

"DataStore can be encrypted via its factories and Jetpack Security" (v1). — DataStore has no built-in encryption, and `security-crypto` (EncryptedFile, EncryptedSharedPreferences) is **deprecated**. Encrypt in a custom serializer with Tink.

INTERVIEW TRAP

Auto Backup restores the encrypted database on a new phone — but Keystore keys don't travel. The app can't open it and crashes. Exclude it from backup, or detect the missing key and resync.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. The KMP data layer

```
// commonMain
@Database(entities = [ArticleEntity::class], version = 1)
@ConstructedBy(AppDatabaseConstructor::class)
abstract class AppDatabase : RoomDatabase() {
    abstract fun articleDao(): ArticleDao
}

// actual is generated by Room's KSP processor
expect object AppDatabaseConstructor :
    RoomDatabaseConstructor<AppDatabase> {
    override fun initialize(): AppDatabase
}

fun createDatabase(
    builder: RoomDatabase.Builder<AppDatabase>,
): AppDatabase = builder
    .setDriver(BundledSQLiteDriver())
    .setQueryCoroutineContext(Dispatchers.IO)
    .build()
```

Each platform only creates the `Builder` with its own file path; everything else is shared.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. What to share, what stays native

Piece	Where it lives
Repositories, mappers, sync and merge logic	`commonMain`
Room entities, DAOs, migrations	`commonMain` (Room KMP)
Preferences	`commonMain` (DataStore KMP, file path per platform)
HTTP	`commonMain` (Ktor), engine per platform: OkHttp / Darwin
Scheduling the sync	Native: WorkManager on Android, BGTaskScheduler on iOS
Keys and secrets	Interface in common; Keystore / Keychain behind it

INTERVIEW TRAP

Blocking DAO functions in common code. Room KMP requires DAO functions in `commonMain` to be `suspend` or return a reactive type like `Flow`.

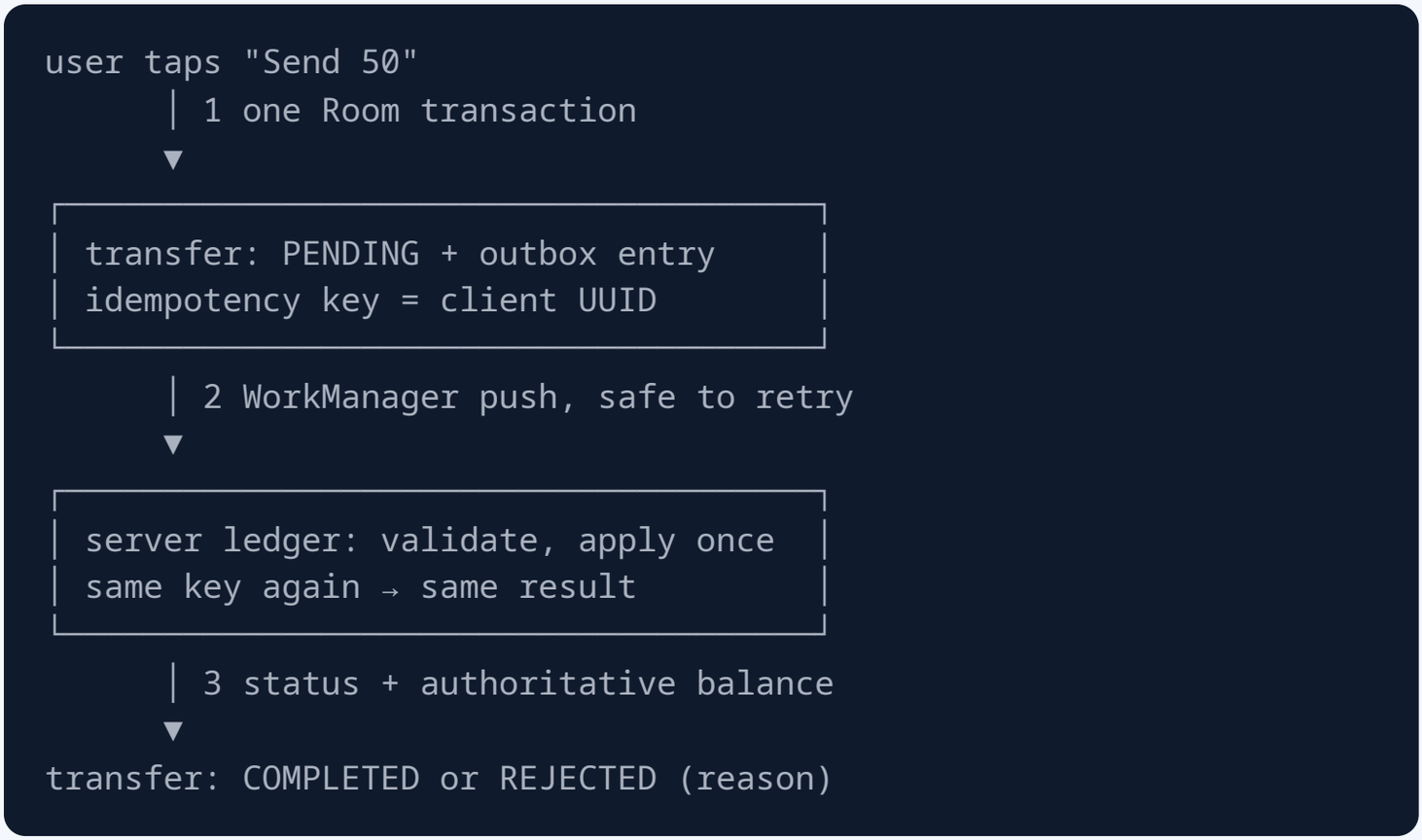
INTERVIEW TRAP

"KMP shares the background sync too." — The sync **logic** is shared; **when** it runs is decided by each OS. Share the `Syncer`, schedule it natively.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Sync for money: an end-to-end design





Esmaeeil Moradi
@esmaeeil-moradi700

Q13. The rules behind the diagram

- The client never edits a balance; it sends **commands**. The server's ledger is the authority
- Show the last confirmed balance **with its time**, and pending transfers separately
- Timeout after sending? Don't guess — retry with the **same key**, or ask the server for the status of that key
- Protect the local copy: encrypted DB, wipe on logout, no amounts or tokens in logs
- Observe the pipeline: outbox depth, age of the oldest pending op, rejection reasons

INTERVIEW TRAP

"Let the user resolve conflicting transactions" (v1). — Money isn't merged. Two transfers that overdraw the account are not a conflict to pick from: the server **rejects** one with a reason, and the app shows it.

SENIOR ANSWER IN ONE LINE

"Read from the local database, write locally with an outbox, sync with idempotent commands in WorkManager, detect conflicts with server versions, resolve them per data type — and test every migration."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 14 · ARCHITECT

Day 14 — cheat sheet

- Offline-first: the UI reads only the local DB; **writes** are accepted offline and synced later.
- Sync state is data: `PENDING` / `SYNCED` / `FAILED` in the row. Client-generated IDs.
- Row + outbox entry in **one transaction**; every operation carries an idempotency key.
- WorkManager: unique work, `CONNECTED` for user writes, `retry()` for transient, `FAILED` for 4xx.
- Pull deltas with a cursor stored **in the same DB**; tombstones for deletions.
- Paging 3: UI pages from Room; `RemoteMediator` fills Room; clear + insert in one transaction.
- Detect conflicts with server versions (`If-Match`), not device clocks; resolve per data type.
- Migrations: exported schemas, every path tested, set-based SQL, no destructive fallback for user data.
- Encryption at rest: Keystore-wrapped keys, SQLCipher/Tink; `security-crypto` is deprecated; mind backups.
- KMP: share Room, DataStore, Ktor and sync logic; schedule natively.

Day 15 is next → Background Work Part 1: WorkManager, Doze & background limits. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700